

8086 - Instruções de saltos

As instruções de salto (ou *jumps*) no microprocessador 8086 servem para alterar o fluxo de execução do programa, desviando o ponteiro de instrução (IP) para um novo endereço de memória (um rótulo ou *label*).

Eles são divididos essencialmente em três categorias: **Incondicional**, **Condicionais** e de **Malha (Loops)**.

1. Salto Incondicional

Não depende de nenhuma condição ou estado das flags. Ao ser encontrado, o desvio é feito imediatamente.

- **JMP (Jump)**: Desvia a execução para o rótulo especificado. Pode ser curto (*short*), perto (*near* - mesmo segmento) ou longe (*far* - segmento diferente).

2. Saltos Condicionais (Baseados em Flags)

Estes saltos analisam o registrador de **FLAGS** (geralmente modificado logo antes por instruções como CMP, SUB, ADD ou TEST) para decidir se realizam ou não o desvio.

A. Para Comparações de Números Sem Sinal (Unsigned)

Usam termos como *Above* (Acima) e *Below* (Abaixo).

- JA / JNBE: Salta se for maior/acima (*Jump if Above*).
- JAE / JNB: Salta se for maior ou igual / não for abaixo.
- JB / JC / JNAE: Salta se for menor/abaixo (*Jump if Below*) ou se houver *Carry* ($CF=1$).
- JBE / JNA: Salta se for menor, igual ou abaixo.

B. Para Comparações de Números Com Sinal (Signed)

Usam termos como *Greater* (Maior) e *Less* (Menor).

- JG / JNLE: Salta se for maior (*Jump if Greater*).
- JGE / JNL: Salta se for maior ou igual.
- JL / JNGE: Salta se for menor (*Jump if Less*).
- JLE / JNG: Salta se for menor ou igual.

C. Baseados em Flags Específicas de Status

- JZ / JE: Salta se o resultado foi zero ou igual ($ZF=1$).
- JNZ / JNE: Salta se o resultado não foi zero ou diferente ($ZF=0$).
- JS: Salta se o resultado for negativo / tem sinal ($SF=1$).
- JNS: Salta se o resultado for positivo / não tem sinal ($SF=0$).

- **JO**: Salta se ocorreu estouro de capacidade em conta com sinal ($OF=1$).
- **JNO**: Salta se não ocorreu estouro ($OF=0$).
- **JP / JPE**: Salta se a paridade for par ($PF=1$).
- **JNP / JPO**: Salta se a paridade for ímpar ($PF=0$).

D. Condição Especial de Registrador

- **JCXZ**: Salta especificamente se o registrador contador **CX** for igual a zero. Muito usado antes de iniciar loops para evitar que girem 65536 vezes caso **CX** comece zerado.

3. Instruções de Controle de Loop (Laço)

São variações especiais de saltos condicionais que decrementam automaticamente o registrador **CX** a cada passagem e saltam se a condição for mantida.

- **LOOP**: Decrementa **CX** em 1. Salta para o rótulo se $CX \neq 0$.
- **LOOPE / LOOPZ**: Decrementa **CX**. Salta se $CX \neq 0$ **E** a flag de zero estiver ativa ($ZF=1$).
- **LOOPNE / LOOPNZ**: Decrementa **CX**. Salta se $CX \neq 0$ **E** a flag de zero não estiver ativa ($ZF=0$).

Para construir uma estrutura **IF / ELSE** no assembly do 8086, você deve utilizar uma instrução de comparação (geralmente **CMP**) seguida de um **salto condicional** para desviar o fluxo caso a condição seja falsa.

Como o assembly executa o código linha por linha (de cima para baixo), você também precisa de um **salto incondicional** (**JMP**) no final do bloco **IF** para "pular" o bloco **ELSE**.

A Lógica Estrutural

Pense na lógica de forma invertida: **"Se a condição não for atendida, pule para o ELSE"**.

1. Compara os valores (**CMP**)
2. Se a condição falhar -> Salta para o rótulo **ELSE**
3. [Código do bloco **IF** executa aqui]
4. Salto incondicional (**JMP**) -> Vai para o **FIM** (ignora o **ELSE**)
5. Rótulo **ELSE**:
6. [Código do bloco **ELSE** executa aqui]
7. Rótulo **FIM**:
8. [Continuação do programa]

Exemplo Prático em Código (emu8086)

Vamos traduzir o seguinte código de alto nível para assembly do 8086:

c:

```

if (AX == BX) {
    CX = 1;    // Bloco IF
} else {
    CX = 2;    // Bloco ELSE
}

```

Código correspondente em Assembly:

assembly:

```

; 1. Compara os valores de AX e BX
CMP AX, BX

; 2. Se NÃO forem iguais (Not Equal), salta para o bloco ELSE
JNE SE_DIFERENTE

; 3. Bloco IF (Executa se AX for IGUAL a BX)
MOV CX, 1

; 4. Pula o bloco ELSE para não executar as duas instruções
seguidas
JMP FIM_IF

SE_DIFERENTE:
    ; 5. Bloco ELSE (Executa se AX for DIFERENTE de BX)
    MOV CX, 2

FIM_IF:
; 6. O programa continua a execução normal a partir daqui

```

Outro Exemplo: Menor ou Igual (Com Sinal)

Se quisermos testar se AX <= BX:

assembly:

```

CMP AX, BX
JG BLOCO_ELSE    ; Se AX for MAIOR (Greater), a condição falhou.
                  Salta para o ELSE.

; --- Bloco IF (AX <= BX) ---
MOV DX, 5
JMP FIM_COMPARA

BLOCO_ELSE:
; --- Bloco ELSE (AX > BX) ---
MOV DX, 10

FIM_COMPARA:

```

Para construir laços de repetição como **WHILE** ou **FOR** no assembly do 8086 sem depender exclusivamente da instrução padrão **LOOP**, utilizamos uma combinação de instruções de comparação (**CMP**), saltos condicionais (como **JNE**, **JGE**, **JL**) e saltos incondicionais (**JMP**).

Abaixo, veja a estrutura lógica e os exemplos práticos de como implementar ambos os laços.

1. Construindo um Laço **WHILE**

O laço **WHILE** avalia a condição **no início**. Se a condição for falsa logo de primeira, o bloco de código de dentro do laço nunca é executado.

Lógica em C/C++:

c:

```
while (AX < 10) {
    AX++; // Código interno
}
```

Código equivalente em Assembly 8086:

assembly:

```
INICIO_WHILE:
    ; 1. Avalia a condição: compara AX com 10
    CMP AX, 10

    ; 2. Se AX for maior ou igual (Greater or Equal) a 10, sai do
    laço
    JGE FIM_WHILE

    ; 3. Bloco de código interno (ex: incrementa AX)
    INC AX

    ; 4. Salta de volta para o início para testar a condição de
    novo
    JMP INICIO_WHILE

FIM_WHILE:
    ; O programa continua a partir daqui quando AX atingir 10
```

2. Construindo um Laço **FOR**

O laço `FOR` possui três partes definidas: a **inicialização** de uma variável contadora, a **condição de parada** e o **passo de incremento/decremento** (geralmente executado no final do bloco).

Lógica em C/C++:

c

```
for (CX = 0; CX < 5; CX++) {  
    // Código interno (ex: soma BX com DX)  
    BX = BX + DX;  
}
```

Código equivalente em Assembly 8086:

assembly

```
; 1. Inicialização: zera o registrador contador (CX = 0)  
XOR CX, CX
```

INICIO_FOR:

```
; 2. Teste da condição: compara o contador CX com o limite 5  
CMP CX, 5
```

```
; 3. Se CX for igual ou maior a 5, encerra o laço  
JGE FIM_FOR
```

```
; 4. Bloco de código interno do laço  
ADD BX, DX
```

```
; 5. Passo: incrementa o contador (CX++)  
INC CX
```

```
; 6. Volta para testar a condição novamente  
JMP INICIO_FOR
```

FIM_FOR:

```
; Continuação do programa após rodar 5 vezes
```

Por que usar saltos em vez da instrução `LOOP`?

Embora a instrução `LOOP` seja mais curta, usar `CMP` + saltos traz vantagens cruciais:

- **Flexibilidade:** A instrução `LOOP` decrementa obrigatoriamente de 1 em 1 e aceita apenas o registrador `CX`. Com saltos, você pode usar qualquer registrador (`AX`, `BX`, `SI`) e incrementar/decrementar pelo valor que quiser (ex: `ADD SI, 2` para percorrer um vetor de Words).
- **Escopo maior:** A instrução `LOOP` aceita apenas saltos curtos (*short jumps* de até 128 bytes de distância na memória). Com saltos genéricos, seu bloco de código interno pode ser muito maior.